No-Brand Vector Machine

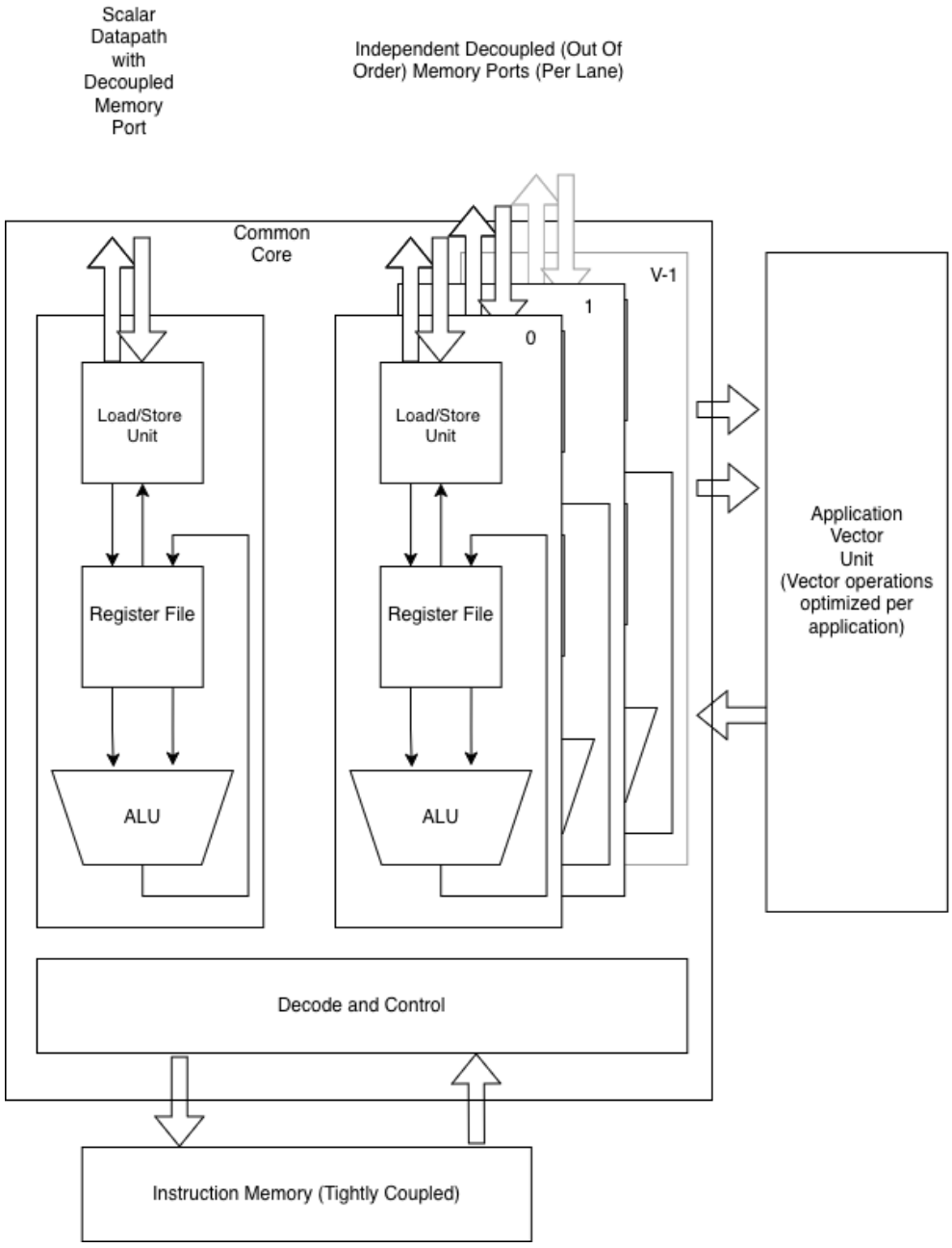
A Generic Vector Core with Application-Specific Vector Acceleration

- Rationale: to fill the gap between off-the-shelf vector DSPs and custom hardware accelerators when computational complexity meets with the desire to express the functionality of a system in software while still being economical in hardware.
- Load/Store machine with 32-bit instruction word and a large architectural register file (max 64 scalar and 64 vector registers, can be scaled down per application)
- Simple static pipeline: fetch, decode, execute/calculate EA, issue load/store
- Generic vector length (practical values 4/8/16/32)
- Generic word length (practical limits 32... 128 bits)
- Data memory ports are decoupled and support out of order completion
- One data memory port for scalar data path + one for each vector lane
- Load/Store units execute memory transactions asynchronously, hardware interlocking provided by a scoreboard
- Vector load/store operations support variable stride and “load/store multiple” (2, 4 or 8 vectors)
- Data memory is byte-addressable
- Predicated execution for vector operations on a sub-word (8 bit) basis

- Generic integer ALU supports basic arithmetic and logic including saturating sub-word (8/16 bits) arithmetic, no cross-lane operations
- Complex or application-specific operations implemented inside an Application Vector Unit, e.g.
    - FP16 complex vector arithmetic, one complex number per 32-bit lane
    - LDPC message-passing decoder with the AVU handling the compares and maintaining the intermediate result
    - Cross-lane operations when needed
- AVU operations can be pipelined and can complete out of order, interlock provided by the scoreboard
- Asynchronous load/store multiple execution can occur in parallel with computation
- Application Example: A Vector Unit with FP16 complex number arithmetic (multiply, add, 3-stage pipeline) and vector length = 16 can implement 4096 point FFT (radix-4) in a bit over 7000 clock cycles, or < 2 clock cycles per point (assuming minimal contention for memory banks and latency < ~10 clock cycles) => real-time 4096-point FFT on a 100Msamples/s stream is possible at modest clock rates under 200MHz
- Core instruction set is defined as a YAML file that drives both instruction decode logic generation and assembler operation, easy to modify
- Assembler + linker exist
    - LLVM C compiler backend is planned: the architecture is an ideal target for C compiler including SIMD support (unlike GPUs)
    - Vectorizing C code for a clean SIMD design is a solved problem - no fancy vendor-specific compiler technology needed (think CUDA)
- RTL VHDL hardware implementation exists

- FPGA demonstrator planned

- Future work: hardware support for multithreading to help applications where a single kernel can't easily utilise a big register file
  - Threads share the physical register file, each thread has a segment
- Block diagram



- Hardware complexity estimate
  - Small 32-bit configuration, 16 registers, V = 4: ~3600 D-flipflops
  - Large 32-bit configuration: 64 registers, V = 16: ~38400 D-flipflops
- Assuming total 30 gates/flipflop including combinational logic:
  - Small configuration (16 registers, V = 4): area ~110,000 gates
  - Large 32-bit configuration (64 registers, V=16): area ~1.2 million gates
  - Large 64-bit configuration (64 registers, V=16): area ~2.2 million gates

Code example: radix-4 Fast Fourier Transform making use of complex vector operations implemented in an AVU (instructions beginning with “cv”) and unrolled by 4 to expose parallelism for memory transfer vs compute:

```
////////////////////////////////////////////////////////////////////////////////
//
// Copyright (c) 2026 Juhani Vehvilainen. All rights reserved.
//
// PROPRIETARY AND CONFIDENTIAL.
// Unauthorized copying, modification, or distribution of this file,
// via any medium, is strictly prohibited.
//
////////////////////////////////////////////////////////////////////////////////

  .export "fft_dit_radix4_unroll4"
  .import "w_table"

#include <VU102.h>
#include <common.h>

  .section "code"

#if EFF_V == 16
#define EFF_LOG4V  2
#elif EFF_V == 64
#define EFF_LOG4V  3
#endif

#ifdef EFF_LOG4V
fft_dit_radix4_unroll4:
  debug 48

  prologue(s3)
```

```
  // Params: s1 = data, s2 = log4(N)

#define _A          s1
#define _LOG4N      s2

  // Full outer loop is
  //   for stage = 1 .. log4(N):
  //
  // Early stage condition:
  //   quarter < V
  //   <==> span / 4 < V
  //   <==> span < 4*V
  //   <==> 4^s < 4*V
  //   <==> s < LOG4(4*V)
  //   <==> s < 1 + LOG4(V)
  //

  // for(s = 1; s <= EFF_LOG4V; s++) {
  //    span = 1<<(2 * s);  // 4^s
  //    quarter = span / 4;
  //    index_mult = FFT_LEN / span;

#define _s           s3
#define _span        s4
#define _quarter     s5
#define _index_mult  s6
#define _W           s7
#define _fft_len     s8

  ldi _s, 1               // s = 1
  load_big(_W, w_table)
  lsl _fft_len, _LOG4N, 1
  lsl _fft_len, _s, _fft_len // fft_len = 1<<(2*log4(N))

for_s_loop_early:
  lsl _span, _s, 1          // span = 2*s
  ldi _quarter, 1
  lsl _span, _quarter, _span   // span = 1<<(2*s) = 4^s
  lsr _quarter, _span, 2    // quarter = span/4
  lsl _index_mult, _s, 1    // index_mult = 2*s
  lsr _index_mult, _fft_len, _index_mult  // index_mult = FFT_LEN/4^s = FFT_LEN/span

  // early stage: small blocks, broadcast twiddle, strided across blocks
  // for j = 0 .. quarter-1:
  //
  // for(j = 0; j < quarter; j++) {

#define _j          s9

  clear(_j)

for_j_loop_early:
```

```
        // Pseudo: w1j, w2j, w3j = twid1[j], twid2[j], twid3[j]   // scalars
        //
        // C: complex_t w1j = w[j * index_mult], w2j = w[j * 2*index_mult], w3j = w[j *
3*index_mult];
        //    int kb = 0;

      #define _w1j       s10
      #define _w2j       s11
      #define _w3j       s12
      #define _delta     s13
      #define _2delta    s14
      #define _temp      s15

        // j * index_mult = j * FFT_LEN/4^s = j<<(2*log4(N))>>(2*s) = j<<(2*(log4(N) - s))

        sub _delta, s2, _s         // delta = log4(N) - s
        lsl _delta, _delta, 1      // delta = 2*(log4(N) - s)
        lsl _delta, _j, _delta  // delta = j<<(2*(log4(N) - s)) = j * index_mult

        ldx _w1j, _W, _delta  // w1j = W[j * mult]
        lsl _temp, _delta, 1           // temp = 2 * j * mult
        ldx _w2j, _W, _temp   // w2j = W[2 * j * mult]
        add _temp, _temp, _delta    // temp = 3 * j * mul
        ldx _w3j, _W, _temp   // w3j = W[3 * j * mult]

        lsl _delta, _quarter, 2     // delta = 4 * quarter
        lsl _2delta, _delta, 1      // 2delta = 8 * quarter

        // for(kb = 0; kb < FFT_LEN; kb += V*span) {
        //      printf("---\ns = %d j = %d, kb = %d\n---\n", s, j, kb);

      #define _kb        s16
      #define _4kbj      s17
      #define _V_x_span  s18
      #define _w1        v0
      #define _w1_2      v1
      #define _w1_3      v2
      #define _w1_4      v3
      #define _w2        v4
      #define _w2_2      v5
      #define _w2_3      v6
      #define _w2_4      v7
      #define _w3        v8
      #define _w3_2      v9
      #define _w3_3      v10
      #define _w3_4      v11
      #define _a0        v16
      #define _a0_2      v17
      #define _a0_3      v18
      #define _a0_4      v19
      #define _a1        v20
```

```
#define _a1_2      v21
#define _a1_3      v22
#define _a1_4      v23
#define _a2        v24
#define _a2_2      v25
#define _a2_3      v26
#define _a2_4      v27
#define _a3        v28
#define _a3_2      v29
#define _a3_3      v30
#define _a3_4      v31
#define _t0        v32
#define _t0_2      v33
#define _t0_3      v34
#define _t0_4      v35
#define _t1        v36
#define _t1_2      v37
#define _t1_3      v38
#define _t1_4      v39
#define _t2        v40
#define _t2_2      v41
#define _t2_3      v42
#define _t2_4      v43
#define _t3        v44
#define _t3_2      v45
#define _t3_3      v46
#define _t3_4      v47
#define _s0        v48
#define _s0_2      v49
#define _s0_3      v50
#define _s0_4      v51
#define _s1        v52
#define _s1_2      v53
#define _s1_3      v54
#define _s1_4      v55
#define _s2        v56
#define _s2_2      v57
#define _s2_3      v58
#define _s2_4      v59
#define _s3        v60
#define _s3_2      v61
#define _s3_3      v62
#define _s3_4      v63
#define _y0        v16
#define _y0_2      v17
#define _y0_3      v18
#define _y0_4      v19
#define _y1        v20
#define _y1_2      v21
#define _y1_3      v22
#define _y1_4      v23
#define _y2        v24
```

```
#define _y2_2      v25
#define _y2_3      v26
#define _y2_4      v27
#define _y3        v28
#define _y3_2      v29
#define _y3_3      v30
#define _y3_4      v31

  clear(_kb)
  lsl _V_x_span, _span, EFF_LOG2V  // Vspan = span<<LOG2V = V * span

for_kb_loop:
#ifdef DEBUG
  debug 0xB00B
  lsl s50, _s, 16
  or s50, s50, _j
  debugr s50
  debugr _kb
#endif

  add _4kbj, _kb, _j    // 4kbj = kb + j
  lsl _4kbj, _4kbj, 2      // 4kbj = 4*(kb + j)
  add _temp, _A, _4kbj  // temp = &A[kb + j]
  add _temp, _temp, _delta // temp = &A[kb + j + quarter]
  vld.4 _a1, _temp, _span    // complex_vector_read(a1, A, kb + j + quarter, span)

  add _temp, _temp, _delta // temp = &A[kb + j + 2*quarter]
  vld.4 _a2, _temp, _span    // complex_vector_read(a2, A, kb + j + 2*quarter, span)

  add _temp, _temp, _delta // temp = &A[kb + j + 3*quarter]
  vld.4 _a3, _temp, _span    // complex_vector_read(a3, A, kb + j + 3*quarter, span)

  add _temp, _A, _4kbj  // temp = &A[kb + j]
  vld.4 _a0, _temp, _span // complex_vector_read(a0, A, kb + j, span)

  cvbmul _t1, _a1, _w1j
  cvbmul _t1_2, _a1_2, _w1j
  cvbmul _t1_3, _a1_3, _w1j
  cvbmul _t1_4, _a1_4, _w1j
  cvbmul _t2, _a2, _w2j
  cvbmul _t2_2, _a2_2, _w2j
  cvbmul _t2_3, _a2_3, _w2j
  cvbmul _t2_4, _a2_4, _w2j
  cvbmul _t3, _a3, _w3j
  cvbmul _t3_2, _a3_2, _w3j
  cvbmul _t3_3, _a3_3, _w3j
  cvbmul _t3_4, _a3_4, _w3j

  cvadd _s0, _a0, _t2
  cvadd _s0_2, _a0_2, _t2_2
  cvadd _s0_3, _a0_3, _t2_3
  cvadd _s0_4, _a0_4, _t2_4
```

```
cvsub _s1, _a0, _t2
cvsub _s1_2, _a0_2, _t2_2
cvsub _s1_3, _a0_3, _t2_3
cvsub _s1_4, _a0_4, _t2_4
cvadd _s2, _t1, _t3
cvadd _s2_2, _t1_2, _t3_2
cvadd _s2_3, _t1_3, _t3_3
cvadd _s2_4, _t1_4, _t3_4
cvsub _s3, _t1, _t3
cvsub _s3_2, _t1_2, _t3_2
cvsub _s3_3, _t1_3, _t3_3
cvsub _s3_4, _t1_4, _t3_4

cvadd _y0, _s0, _s2
cvadd _y0_2, _s0_2, _s2_2
cvadd _y0_3, _s0_3, _s2_3
cvadd _y0_4, _s0_4, _s2_4

cvsub _y2, _s0, _s2
cvsub _y2_2, _s0_2, _s2_2
cvsub _y2_3, _s0_3, _s2_3
cvsub _y2_4, _s0_4, _s2_4

add _temp, _A, _4kbj  // temp = &A[kb + j]
vst.4 _y0, _temp, _span // complex_vector_write(y0, A, kb + j, span)

cvaddrni _y1, _s1, _s3
cvaddrni _y1_2, _s1_2, _s3_2
cvaddrni _y1_3, _s1_3, _s3_3
cvaddrni _y1_4, _s1_4, _s3_4

add _temp, _temp, _2delta // temp = &A[kb + j + 2*quarter]
vst.4 _y2, _temp, _span    // complex_vector_write(y2, A, kb + j + 2*quarter, span)

cvaddri _y3, _s1, _s3
cvaddri _y3_2, _s1_2, _s3_2
cvaddri _y3_3, _s1_3, _s3_3
cvaddri _y3_4, _s1_4, _s3_4

sub _temp, _temp, _delta // temp = &A[kb + j + quarter]
vst.4 _y1, _temp, _span    // complex_vector_write(y1, A, kb + j + quarter, span)

add _temp, _temp, _2delta // temp = &A[kb + j + 3*quarter]

// end for(kb = 0; kb < FFT_LEN; kb += V*span)

add _kb, _kb, _V_x_span     // kb += V * span
cmp c0, _kb, _fft_len       // c0 = compare(kb, fft_len)

blt c0, for_kb_loop               // while kb < fft_len
vst.4 _y3, _temp, _span    // complex_vector_write(y3, A, kb + j + 3*quarter, span)
```

```
  //
  // end for(j = 0; j < quarter; j++)
  //

  add _j, _j, 1
  cmp c0, _j, _quarter
  blt c0, for_j_loop_early
  nop

  //
  // end for(s = 1; s <= EFF_LOG4V; s++)
  //

  add _s, _s, 1
  cmp c0, _s, EFF_LOG4V
  ble c0, for_s_loop_early
  drain      // Store fence

  // for(s = LOG4V + 1; s <= log4(N); s++) {

for_s_loop_late:
  // late stage: contiguous within a block, twiddle vectors
  //
  // span    = 4^stage
  // quarter = span / 4

  lsl _span, _s, 1          // span = 2*s
  ldi _quarter, 1
  lsl _span, _quarter, _span   // span = 1<<(2*s) = 4^s
  lsr _quarter, _span, 2    // quarter = span/4
  lsl _delta, _quarter, 2     // delta = 4 * quarter
  lsl _2delta, _delta, 1     // 2delta = 8 * quarter
  lsl _index_mult, _s, 1    // index_mult = 2*s
  lsr _index_mult, _fft_len, _index_mult  // index_mult = FFT_LEN/4^s = FFT_LEN/span

#define _k              s16
#define _4kj            s17
#define _delta_j        s18
#define _delta_delta_j  s19
#define _w_stride       s20

  // Pseudo: for j = 0, V, 2V, ... quarter-V:
  //
  // for(j = 0; j < quarter; j += V) {

  clear(_j)

  //
  // delta_j = j * index_mult = j * fft_len>>(2*s)
  // Because j increments by V:
  // delta_delta_j = V * fft_len>>(2**s)
  //   = (1<<LOG2V) * fft_len>>(2*s)
```

```
        //   = (1<<LOG2V) * ((1<<(2*log4(N)))>>(2*s))
        //   = (1<<LOG2V) * (1<<(2*(log4(N) - s)))
        //   = 1<<(LOG2V + 2*(log4(N) - s))
        //

        sub _delta_delta_j, _LOG4N, _s
        lsl _delta_delta_j, _delta_delta_j, 1
        add _delta_delta_j, _delta_delta_j, EFF_LOG2V
        ldi _temp, 1
        lsl _delta_delta_j, _temp, _delta_delta_j
        lsl _delta_delta_j, _delta_delta_j, 2     // Multiply by 4 for a byte offset!!

        clear(_delta_j)

      for_j_loop_late:
      #ifdef DEBUG
        debug 0xdada
        lsl s50, _s, 16
        or s50, s50, _j
        debugr s50
      #endif

        add _temp, _W, _delta_j
        move(_w_stride, _index_mult)
        vld.4 _w1, _temp, _w_stride // complex_vector_read(w1, w, j * index_mult, index_mult)

        add _temp, _temp, _delta_j
        add _w_stride, _w_stride, _index_mult
        vld.4 _w2, _temp, _w_stride // complex_vector_read(w2, w, j * 2*index_mult,
2*index_mult)

        add _temp, _temp, _delta_j
        add _w_stride, _w_stride, _index_mult
        vld.4 _w3, _temp, _w_stride // complex_vector_read(w3, w, j * 3*index_mult,
3*index_mult)

        //
        // Pseudo: for k = 0, span, 2*span, ... N-span:
        //
        // for(k = 0; k < FFT_LEN; k += span)
        //

        clear(_k)

      for_k_loop_late:
      #ifdef DEBUG
        sync
        debug 0xBEEF
        lsl s50, _s, 16
        or s50, s50, _j
        debugr s50
        debugr _kb
```

```
#endif

  add _4kj, _k, _j    // 4kj = k + j
  lsl _4kj, _4kj, 2      // 4kj = 4*(k + j)

  add _temp, _A, _4kj  // temp = &A[k + j]
  add _temp, _temp, _delta // temp = &A[k + j + quarter]
  vld.4 _a1, _temp, 1 // complex_vector_read(a1, A, k + j + quarter, 1)

  add _temp, _temp, _delta // temp = &A[k + j + 2*quarter]
  vld.4 _a2, _temp, 1 // complex_vector_read(a2, A, k + j + 2*quarter, 1)

  add _temp, _temp, _delta // temp = &A[k + j + 3*quarter]
  vld.4 _a3, _temp, 1 // complex_vector_read(a3, A, k + j + 3*quarter, 1)

  add _temp, _A, _4kj  // temp = &A[k + j]
  vld.4 _a0, _temp, 1 // complex_vector_read(a0, A, k + j, 1)

  cvmul _t1, _w1, _a1
  cvmul _t1_2, _w1_2, _a1_2
  cvmul _t1_3, _w1_3, _a1_3
  cvmul _t1_4, _w1_4, _a1_4
  cvmul _t2, _w2, _a2
  cvmul _t2_2, _w2_2, _a2_2
  cvmul _t2_3, _w2_3, _a2_3
  cvmul _t2_4, _w2_4, _a2_4
  cvmul _t3, _w3, _a3
  cvmul _t3_2, _w3_2, _a3_2
  cvmul _t3_3, _w3_3, _a3_3
  cvmul _t3_4, _w3_4, _a3_4

  cvadd _s0, _a0, _t2
  cvadd _s0_2, _a0_2, _t2_2
  cvadd _s0_3, _a0_3, _t2_3
  cvadd _s0_4, _a0_4, _t2_4
  cvsub _s1, _a0, _t2
  cvsub _s1_2, _a0_2, _t2_2
  cvsub _s1_3, _a0_3, _t2_3
  cvsub _s1_4, _a0_4, _t2_4
  cvadd _s2, _t1, _t3
  cvadd _s2_2, _t1_2, _t3_2
  cvadd _s2_3, _t1_3, _t3_3
  cvadd _s2_4, _t1_4, _t3_4
  cvsub _s3, _t1, _t3
  cvsub _s3_2, _t1_2, _t3_2
  cvsub _s3_3, _t1_3, _t3_3
  cvsub _s3_4, _t1_4, _t3_4

  cvadd _y0, _s0, _s2
  cvadd _y0_2, _s0_2, _s2_2
  cvadd _y0_3, _s0_3, _s2_3
  cvadd _y0_4, _s0_4, _s2_4
```

```
cvsub _y2, _s0, _s2
cvsub _y2_2, _s0_2, _s2_2
cvsub _y2_3, _s0_3, _s2_3
cvsub _y2_4, _s0_4, _s2_4

add _temp, _A, _4kj
vst.4 _y0, _temp, 1

cvaddrni _y1, _s1, _s3
cvaddrni _y1_2, _s1_2, _s3_2
cvaddrni _y1_3, _s1_3, _s3_3
cvaddrni _y1_4, _s1_4, _s3_4

add _temp, _temp, _2delta
vst.4 _y2, _temp, 1

cvaddri _y3, _s1, _s3
cvaddri _y3_2, _s1_2, _s3_2
cvaddri _y3_3, _s1_3, _s3_3
cvaddri _y3_4, _s1_4, _s3_4

sub _temp, _temp, _delta
vst.4 _y1, _temp, 1

add _temp, _temp, _2delta

// end for(k = 0; k < FFT_LEN; k += span)

add _k, _k, _span
cmp c0, _k, _fft_len
blt c0, for_k_loop_late
vst.4 _y3, _temp, 1

// end for(j = 0; j < quarter; j += V) {

add _j, _j, EFF_V
cmp c0, _j, _quarter
blt c0, for_j_loop_late
add _delta_j, _delta_j, _delta_delta_j

// end for(s = LOG4V + 1; s <= log4(N); s++) {

add _s, _s, 1
cmp c0, _s, _LOG4N
ble c0, for_s_loop_late
drain      // Store fence

// Done

epilogue(s3)
```

```
  ret
  nop
#endif
```